程序设计综合实验

实验 2：在格子与图中游走

黄韬
taohonker@gmail.com
昆明理工大学·系统科学系

2026-09-30

目录

1 实验简介与要求 . . . 3
1.1 实验简介 . . . 3
1.2 实验要求 . . . 4
2 格子的数组实现与地势优化 . . . 4
2.1 数组实现格子与地势统计 . . . 5
2.2 使用 Python 制作程序可视化界面 . . . 5
3 复杂地形上的随机行走 . . . 6
3.1 随机行走规则 . . . 6
3.2 随机行走程序 . . . 7
3.3 检查点 1 . . . 7
4 格子的链状节点实现 . . . 8
4.1 链状节点构成方格子 . . . 8
4.2 检查点 2 . . . 9
5 地上有陷阱：栈与队列的综合练习 . . . 9
5.1 检查点 3 . . . 10
5.2 任务描述 1. 陷阱机制 . . . 10
5.3 任务描述 2. 平地规则 . . . 11
5.4 任务描述 3. 队列机制 . . . 11
5.5 实现要求 . . . 12
5.6 提交要求 . . . 12
5.7 检查点 3 的意义与背景 . . . 13
6 多体游走 . . . 14
6.1 多个 walker 的相互排斥与回避 . . . 14
6.2 检查点 4 . . . 15
7 寻路 . . . 15
7.1 最短路径 . . . 16
7.2 检查点 5 . . . 16

实验简介与要求

实验简介

- 这是一个**将延续整个学期的长期实验**，围绕格子与图上的随机行走为中心话题，实验内容涉及数组操作、指针、排序、搜索、图论、优化算法、动力学与统计、数据处理与可视化、设计模式等。
- 实验分若干检查点，每个检查点单独建仓库、单独时限；提交一律是 **fork 对应仓库后推送到自己的 fork**，不建 Pull Request / Merge Request。到期末时做最后的整理与提交（整理仓库届时公布）。
- 实验内容会随着对应理论课程的进程逐步增加。
- 能够完成所有检查点并知道自己在做什么的同学，两年半后如果想在我的科研组实习或读研，可以优先考虑。
- 实验中的相关概念与技能对于很多科研方向都非常重要，建议认真完成。

实验要求

- 提交形式：markdown + 代码。每个检查点对应课程组织下的一个独立仓库，**fork 不改名**，在自己的 fork 上开发并推送；评分只看自己 fork 仓库内的 Git 记录与推送历史，不建 PR/MR。
- 本次（2026-09-30）布置检查点 1 与检查点 2，都在仓库 `dsa2026/lab02-checkpoint01_02` 提交：检查点 1 的成果放自己 fork 的 `checkpoint1/` 目录，检查点 2 放 `checkpoint2/` 目录。
- **截止时间：2026 年 10 月 13 日（周二）晚 23:59**，以截止前自己 fork 上的最后一次推送为准。
- 代码应清晰规范，注释恰当，变量命名规范
- 报告在前几次提交时简要即可
- 每次提交形成独立的目录、代码与简要报告
- 最终提交完整报告与代码
- **提醒**：每个检查点在满足要求情况下，尽量把程序改得更灵活通用，后续实验才能平稳实现。如果每次都只是针对当前检查点完成实验，后面的问题会越来越难以完成。

格子的数组实现与地势优化

数组实现格子与地势统计

1. 在 C 语言中，初始化一个 100 * 100 的数组作为方格子，每个格点赋值为 $[0, 1)$ 之间均匀分布的随机数。这个过程写成函数，**返回数组**。
2. 一个格点 i 的**局域粗糙度** ρ_i 定义为一个点与其相邻点的值的绝对差之和。写成函数计算这个值，**返回局域粗糙度**。

   $$\rho_i \equiv \sum_{j \in \partial i} |v_i - v_j|$$

   其中 v_i 为格点 i 的地形高度，∂i 为格点 i 的相邻格点。
3. 方格子的**全局粗糙度**定义为所有格点的局域粗糙度之和。写成函数统计这个值，**返回全局粗糙度**。

使用 Python 制作程序可视化界面

1. 在 Python 语言中，初始化一个 101 * 101 的数组作为方格子，每个格点赋值为 $[0, 1)$ 之间均匀分布的随机数
2. 使用 `ctypes` 库，将上述初始化的数组赋值给 C 语言的数组，然后在 Python 中调用 C 语言的函数来计算全局粗糙度
3. 使用 `matplotlib` 库，绘制方格子的地形图（`surface` 函数以及 `contour` 函数），地形图上标明全局粗糙度
4. 重新排列随机数的分布，重复 2-3 步，画出 4 张标明全局粗糙度的地形图：
   i. 将所有生成的随机数重新随机排序（数字的集合不变，但赋值到不同格点上）
   ii. 将所有生成的随机数从小到大排序，依次按从左到右，从上到下的顺序分配给格点
   iii. 将所有生成的随机数从小到大排序，依次按格点坐标和（$v_{i,j}$的坐标和为$i + j$）从小到大的顺序分配给格点
   iv. 将所有生成的随机数从小到大排序，依次按距离中心格点（$v_{51,51}$）的距离从小到大的顺序分配给格点

复杂地形上的随机行走

随机行走规则

1. 假设一个 walker 从方格子中心格点出发，每次以相同的概率选择一个方向：上、下、左、右。
2. 每次选择方向后，是否真的完成行走，取决于当前格点与下一格点的地势差：

$$P_{i->j} = \begin{cases} 1 & \text{if } v_i \geq v_j \,, \\ \exp[(v_i - v_j)/T] & \text{if } v_i < v_j \,. \end{cases}$$

其中 T 是 walker 的“温度”，代表它的行动能力。

随机行走程序

在 C 中：

- 根据指定参数 T，实现在已知地形上的随机行走，并记录每次的行走路径，要求路径长度不小于 1000

在 Python 中：

- 更换不同的 T 与地形，调用 C 程序完成计算，得到路径，并在地形图上绘制路径

检查点 1

- **C**:
  - 利用静态数组实现方格子上的地势赋值
  - 实现完全随机等 4 种地形分布
  - 实现随机行走，记录每次行走的路径
- **Python**:
  - 实现对格子上的地势值的排序与重新分布
  - 向 C 的数组是传递地势值
  - 向 C 的随机行走过程传递参数，驱动随机行走过程的执行
  - 从 C 的随机行走过程中获取结果，做出地势图，并在图上绘制随机行走的路径
- **提交方式与时间**：
  - fork 课程组织 dsa2026 下的 `lab02-checkpoint01_02` 仓库（不改名），成果推送到自己 fork 的 `checkpoint1/` 目录；不建 PR/MR
  - *26-10-13* 晚 23:59 前完成推送，评分以截止前最后一次推送为准

格子的链状节点实现

链状节点构成方格子

需要完成的问题一如第 1、2 节的要求，构建方格子，完成地势相关的统计与在格子上随机游走的动力学模拟并绘图，但方格子的底层实现改为链接节点。

1. 在 C 语言中，
   i. 定义带有四个链接方向的节点，其数据域记录$[0, 1)$ 之间均匀分布的随机数作为节点上的地势值。
   ii. 将节点链接为 101×101 的方格子，边界使用空指针 `Null` 实现，格子的入口为中心节点 $(51, 51)$.
   iii. 完成从数组格子向链状格子的地势转换函数。
   iv. 设计一种能够从入口节点出发、遍历所有节点的方案。
   v. 借助遍历节点的方案，在链状格子上完成地势统计，计算其局域粗糙度与全局粗糙度。
2. 在 C 语言中，完成单个 walker 从入门节点出发的随机游走，并设计一种记录路径的方案。
3. 在 Python 中，完成对基于链状格子的调用与数据交互，重新完成模拟并绘图。

检查点 2

- **C**:
  - 定义带有四个方向指针的节点，将其链接为方格子
  - 在链状方格子与数组方格子之间进行转换
  - 遍历链状格子所有节点
  - 在链状方格子上实现完全随机等 4 种地形分布
  - 在链状方格子上进行地势统计
  - 在链状方格子上实现随机行走，记录每次行走的路径
- **Python**:
  - 借助之前 C 中的数组实现，连接 C 的链状格子与 Python 界面，完成地形与路径数据的传递
  - 向 C 的链状格子传递地势值
  - 向 C 的随机行走过程传递参数，驱动随机行走过程的执行
  - 从 C 的随机行走过程中获取结果，做出地势图，并在图上绘制随机行走的路径
- **提交方式与时间**:
  - 同一个仓库 `lab02-checkpoint01_02`，成果推送到自己 fork 的 `checkpoint2/` 目录；不建 PR/MR
  - *26-10-13* 晚 23:59 前完成推送，评分以截止前最后一次推送为准

地上有陷阱：栈与队列的综合练习

检查点 3

本检查点结合前面格子地形与随机行走的基础，加入**栈（stack）与队列（queue）**的数据结构练习，并引入“陷阱”机制，考察复杂状态下的粒子运动与数据结构操作。

任务描述 1. 陷阱机制

- 在每个格点 (i, j) 上，附加一个“陷阱”，陷阱深度 $d_{i,j}$ 为 $[0, 9]$ 的随机整数。
- 每个陷阱可容纳 $d_{i,j}$ 个粒子（栈结构，先进后出）。
- 当粒子行走到某格点时，若该点陷阱未满，则粒子落入陷阱（压栈）；若已满，则粒子停留在陷阱上方（不进入陷阱，也不堆积）。
- 每一步，陷阱内的粒子（非满或刚满时，且上方无堆积粒子）有一定概率尝试跳出陷阱，概率为

$$P_{\text{out}} = \exp\left(-\frac{\textbf{空位数} + 1}{\textbf{陷阱深度}}.\right)$$

- 跳出陷阱后，粒子随机选择一个方向尝试移动，若目标格点有空位（无粒子、陷阱未满），则跳跃成功；否则粒子回到原陷阱（重新入栈）。

任务描述 2. 平地规则

- 没有陷阱的格点（陷阱深度为 0）或陷阱已满的格点最多只能被一个粒子占据。
- 每一步，平地上的粒子随机选择方向跳跃，若目标格点有空位或陷阱未满，则跳跃成功；否则原地不动。

任务描述 3. 队列机制

- 在格子的中心点 $(\lfloor L_x/2 \rfloor, \lfloor L_y/2 \rfloor)$ 设置一个队列（queue）。
- 每一步，以概率 p_{out} 随机从队列头部出队一个粒子，粒子尝试向随机方向跳跃，若目标格点有空位或陷阱未满，则跳跃成功，否则粒子回到队列尾部。
- 每一步跳跃尝试结束后，以概率 p_{in} 向队列尾部加入一个新粒子（若队列未满）。
- 队列中的每个粒子可有编号、质量等属性（为后续实验做准备）。

实现要求

- **C 语言：**
  - 为每个格点实现陷阱（栈）结构，支持粒子入栈、出栈操作。
  - 在中心点实现队列结构，支持粒子入队、出队操作。
  - 实现粒子的编号、质量等属性的记录与传递。
  - 实现粒子在平地、陷阱、队列之间的完整运动逻辑。
  - 记录每一步粒子的状态变化，便于后续分析与可视化。
- **Python：**
  - 调用 C 程序，传递地图大小、陷阱深度等常量参数数据。地图不小于 40×40。
  - 可视化粒子分布、陷阱状态、队列长度等动态过程。步数不小于 5000 步。分布、状态等可使用热力图、柱状图等形式展示。
  - 支持参数调整与多次实验对比。

提交要求

- **C**:
  - 实现陷阱（栈）与队列的数据结构及相关操作
  - 完成粒子运动与状态更新的主循环
  - 支持粒子属性的扩展
- **Python**:
  - 实现与 C 的数据交互与可视化
  - 展示不同参数下的实验结果
- **提示**
  - 建议优先实现栈与队列的基本操作，再逐步集成到粒子运动主循环中。
  - 注意陷阱与队列的边界条件与特殊情况（如陷阱满、队列满、粒子属性传递等）。
  - 记录和可视化每一步的粒子分布，有助于调试和分析实验结果。
- **提交方式与时间：**
  - 检查点 3 单独建仓库（届时公布仓库名），仍按统一口径：fork 不改名、推送到自己 fork 的 `checkpoint3/` 目录、不建 PR/MR
  - 截止时间预计 2026 年 11 月中旬，以课程公告为准

检查点 3 的意义与背景

本检查点不仅是对**栈（stack）与队列（queue）**两种基础数据结构的编程训练，更是将它们与复杂系统动力学、统计物理中的“陷阱动力学”与“自由能地形（Free Energy Landscape）”等概念结合，体现数据结构在科学建模中的实际价值。

1. **数据结构与复杂系统建模**
   - 通过在格点上引入陷阱（栈）和中心队列，学生需要在真实动力学模拟中灵活设计和操作数据结构，理解“先进后出”“先进先出”等抽象规则如何映射到物理过程（如粒子陷落、逃逸、排队等待）。
   - 这种设计不仅考察基本编程能力，更训练学生将抽象算法与实际问题结合的能力。
2. **动力学与统计物理背景**
   - “陷阱”机制模拟了物理系统中的势阱、障碍、能垒等现象，粒子在陷阱中的进出概率与阱深、空位等参数相关，反映了实际系统中“动力学迟滞”“非平衡过程”等复杂行为。
   - 队列机制则对应于现实中的排队、服务、拥堵等现象，常见于交通、通信、服务系统等领域。

检查点 3 的意义与背景

3. **Free Energy Landscape 上的遍历与统计**
   - 在更高层次上，整个格子地形可以视为一个**自由能地形（Free Energy Landscape）**，每个格点的高度或陷阱深度对应于系统的能量状态。
   - 粒子在地形上的运动、陷阱中的驻留、队列中的等待等过程，实际上就是在自由能地形上的遍历与采样。
   - 通过统计粒子在不同能量区间的分布、跃迁概率、驻留时间等，可以揭示系统的稳定性、动力学瓶颈、稀有事件等物理性质。
   - 这种建模和统计方法广泛应用于材料科学、生物分子动力学、复杂网络等领域，是理解复杂系统行为的重要工具。
4. **为后续实验铺垫**
   - 本检查点要求为粒子设计编号、质量等属性，为后续多粒子相互作用、统计分析、路径优化等实验内容做好准备。
   - 通过本次练习，学生将在实际编程和建模中体会到数据结构设计的意义，提升将抽象算法与现实问题结合的能力，并为后续深入研究复杂系统动力学和统计物理打下基础。

多体游走

多个 walker 的相互排斥与回避

以检查点 1 为背景，增加格子上游走的 walker，它们之间彼此排斥与回避。具体方式如下：

1. **回避原则**：如果两个 walker 相邻，则禁止移动到对方所在格点。
2. **排斥原则**：每个 walker 会对周围 $5 \times 5 - 1 = 24$ 个格子内的其他 walker 之间产生相互作用，其相互作用势的形式为：

$$V(r) = -\frac{e^{-r}}{r}, \quad r \equiv |\mathbf{r}_i - \mathbf{r}_j|.$$

利用 $\mathbf{F}_{ij} = -\nabla_{\mathbf{r}_i} V(r)$ 可以算出彼此的相互作用力。相互作用力的效果为：

- 位移时，按力在位移方向的分量大小改变位移的概率，你可以自己设计具体的影响方式，保证
  - 力越大，位移概率的改变越大；
  - 力为零时，位移概率不变；
  - 概率既不大于 1，也不小于 0.

检查点 4

- 数学/物理：推导力向量的形式，设计力对移动概率的影响方式。
- **C**:
  - 实现多 walkers 在给定地形上的游走
  - 记录多 walkers 的轨迹
- **Python**:
  - 向 C 程序传递 walkers 的初始位置
  - 从 C 的随机行走过程中获取结果，做出地势图，并在图上绘制多 walkers 随机行走的路径
- **提交方式与时间**：
  - 检查点 4 单独建仓库（届时公布），统一口径：fork 不改名、推送到自己 fork 的 `checkpoint4/` 目录、不建 PR/MR
  - 截止时间预计 2026 年 11 月下旬，以课程公告为准

寻路

最短路径

在格子结构上给定随机地形 $\{v_i\}$，$v_i \sim \mathcal{N}(1,1)$. 并给定任意两点 L_0, L_{end}，如果中间经过 $L_i, i \in [1, N]$ 格点能在给定两点间连接出一条路径，则路径长度定义为：

$$L = \sum_{i=0}^{N} L_{i,i+1},$$

其中 $L_{N+1} \equiv L_{\text{end}}$，

$$L_{i,i+1} = \begin{cases} v_{i+1} - v_i, & \text{如果 } v_{i+1} > v_i; \\ 0, & \text{其它情况}. \end{cases}$$

求给定两点间的“最短路径”。

检查点 5

- **提交方式与时间**：检查点 5 单独建仓库（届时公布），统一口径：fork 不改名、推送到自己 fork 的 `checkpoint5/` 目录、不建 PR/MR；截止时间预计 2027 年 1 月初，以课程公告为准
- 任务要求
  - （不限语言）
  1. 对 10×10 大小的地图，使用 Dijkstra 算法严格求解
  2. 统计 Dijkstra 算法的平均求解步数与地图大小的关系
  3. 【选做：使用随机行走算法求近似解，并与 Dijkstra 算法的解进行比较，100%】
  4. 【选做：对更大的地图，设计启发式算法对两点间路径进行长度优化，100%】